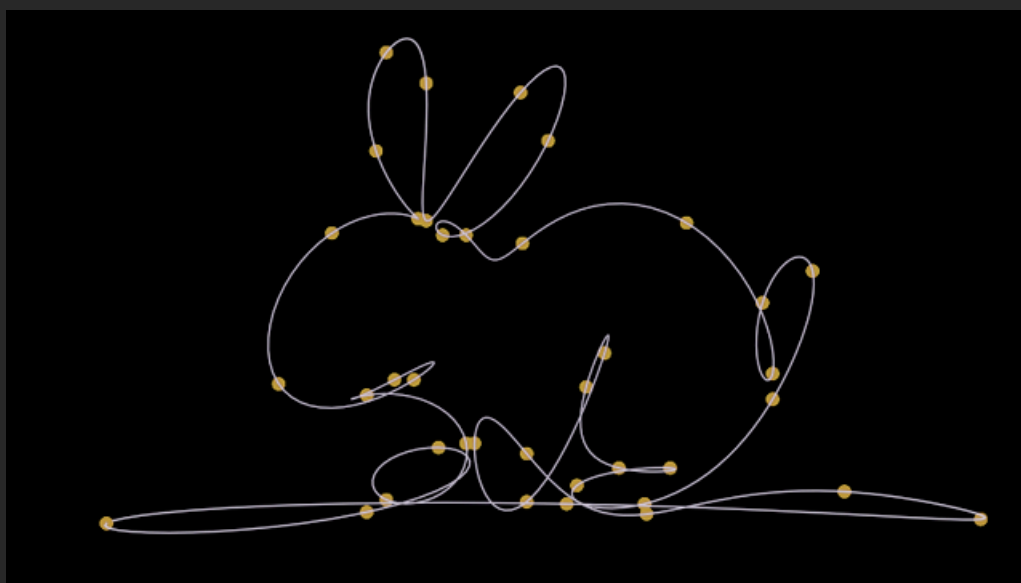


Inigo Quilez :: articles :: fourier series - 2008



Motivation

I was introduced to the Fourier transform and series at the age of 19, and the first thing I thought was, "hey, this can be used for the 3d camera movements in my demos". Originally I was more into regular demo making instead of procedural generation and compression, so at that time I wanted to use the new toy of Fourier for keyframe interpolation, I didn't think of compression. Despite my enthusiasm, my teacher didn't show much interest in the topic of interpolation. Still I programmed it (results down there) but never used it in any demo because splines were cooler then. Anyways, here it goes (much blah blah and little code this time, sorry, I wrote this back then).



38 data points interpolated with Fourier transform

The maths

You might remember from algebra courses the following idea: take a vector (or function or signal) f , and a set of axes φ_n (or "coordinate system" or "basis"), and rewrite f as a linear combination of those axes such that

$$f = \sum_n a_n \varphi_n$$

where the values a_n are called the "coordinates" of the vector f in the new system.

We could choose any coordinate system (Legendre polynomials or simple powers like in Taylor series, you name it). Instead, we take care to choose our axes such that they are orthogonal to each other. Also, we seek a coordinate system where axes have more and more detail as n increases. There is in fact a very popular coordinate system that fulfills these requirements: the trigonometric family of functions (ie, sinus and cosinus series, that I will write as complex exponentials for a more compact representation) given by

$$\varphi_n(x) = e^{jnx}$$

These are the Fourier series (that you probably know already, like many of their cousins - DFT, DCT, Spherical Harmonics, Laplace, etc). Every axis φ_n is perpendicular to each other as can be manually checked by dot-producting any two of the axes:

$$(\varphi_n \cdot \bar{\varphi}_m) = (e^{jnx} \cdot e^{-jmx}) = \frac{1}{2\pi} \int_{-\pi}^{\pi} e^{jnx} e^{-jmx} dx = \delta_{n,m}$$

Now, to get the coordinates a_n (called "Fourier Coefficients" when using the basis), we just have to do:

$$(\bar{\varphi}_m \cdot f) = \bar{\varphi}_m \cdot \sum_n a_n \varphi_n = \sum_n a_n (\bar{\varphi}_m \cdot \varphi_n), \text{ and since we our axes are perpendicular to each other, then we can say}$$

$$(\bar{\varphi}_m \cdot f) = \sum_n a_n \delta_{n,m} = a_m$$

So, computing the coordinates a_n of f is as simple as doing a dot product between the vector and the axes of the new coordinate system (well, their conjugate if they are complex). Just as in regular geometric vector spaces (ie, we are just applying a rotation matrix to our vector f).

The concept

Memory refreshed? Ok, let's go on.

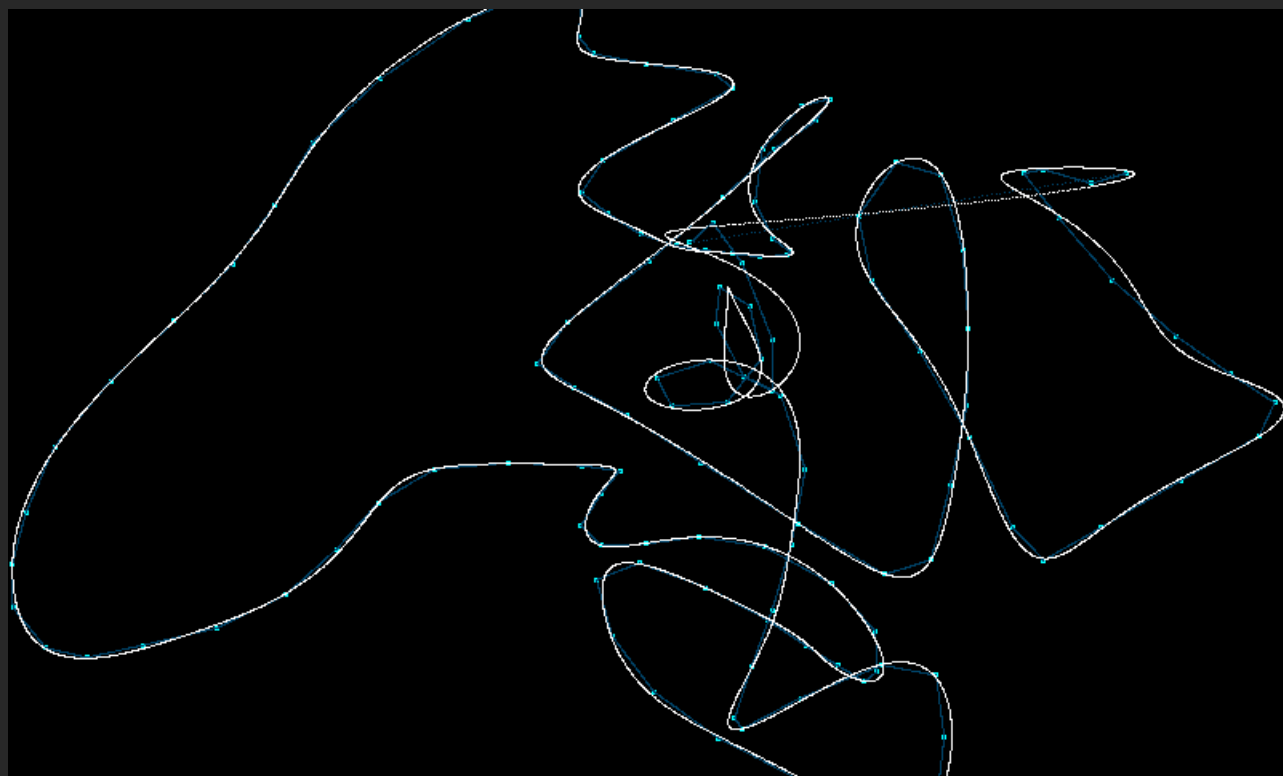
Now, by rotating the vector f by the matrix $\varphi_{n,m}$ we have transformed its representation, ie, now it's a_n what we have to store in our demo, not f anymore. As we said, the higher order axes φ_n represent the detail structure of f , while the global structure of f gets mostly projected into the lower order axes. Assuming f will represent real world data, like camera movements or shapes, it will probably be quite ok if we only pay attention to its global shape, and we discard the rest of the information (for efficient storage and compression purposes). That

means we only need to store few of the coordinates α_n . That's what JPL images or MP3 music do, so it should work for our camera paths too.

The experiments

So I quickly coded a proof of concept. I generated a random two dimensional set of 50 points. Then I computed two Fourier series, one for the x coordinate and another for the y . Each of the Fourier series gives 50 coefficients α_n of course. When using all of them to reconstruct a signal the resulting path exactly passes through the 50 points. Perfect! That's a good way to do camera interpolation (think of the 50 points as key frames).

Next the experiment was to start removing coefficients and see what happened. This is an image using only 80% of the coefficients (40):



and this is what happens when using just 20% of the data (10 coefficients):



Results are not as bad as you would expect, because even the full set of 50 coefficients is very easily compressible, much more than the original description of f . For example, since we know that the last coordinates α_n are going to be smaller than the first, you can actually use less bits for those. Well, there many such tricks, just have a look to the JPEG specification for more inspiration.

The code

This live example in Shadertoy shows the approximation of a shape by an increasing number of Fourier coefficients: <https://www.shadertoy.com/view/ltKSWDM>.

And this one shows the interpolation in action: <https://www.shadertoy.com/view/4lGSDw>

inigo quilez - learning computer graphics since 1994